

P0543R3: Saturation arithmetic

Introduction

Arithmetic on unsigned integers is performed modulo 2^N in C and C++ (6.8.2 [basic.fundamental] p2):

The range of representable values for the unsigned type is 0 to $2^N - 1$ (inclusive); arithmetic for the unsigned type is performed modulo 2^N .

Signed integer operations have undefined behavior when the result is not representable (7.1 [expr.pre] p4):

If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.

In order to implement some algorithms, the use of saturation arithmetic is necessary, where an operation yielding a result whose absolute value is too large instead returns the smallest or largest representable number. For example, when determining the color of a pixel, it would not make sense that brightening a white pixel suddenly turns it black or dark-grey. Instead, brightening a white pixel should simply yield a white pixel.

This paper proposes to add simple free functions for basic saturating operations on all signed and unsigned integer types. Further, a `saturate_cast<T>` is provided that can convert from any of those types to any other, saturating the value as needed.

A previous proposal was in "N3864: A constexpr bitwise operations library for C++" by Matthew Fioravante, but only for addition and subtraction.

It is expected that the functions provided with this proposal will be, at some later time, overloaded for `std::simd`, the nascent SIMD data type (see P0214R2 "Data-Parallel Vector Types & Operations" by Matthias Kretz).

Revision history

R3

- LWG review: Add remark to `div_sat` about "not core constant expression" when the precondition is violated.

R2

- As directed by LEWG: put into `<numeric>` header, mark as freestanding

R1

- Updated section references.
- Addressed SG6 review comments on the reflector.
- Add naming discussion and rename functions to `add_sat` pattern.

Examples

The following examples assume `CHAR_BIT == 8`.

```
int x1 = add_sat(3, 4);           // ok, yields 7
int x2 = sub_sat(INT_MIN, 1);     // ok, yields INT_MIN
unsigned char x3 = add_sat(255, 4); // compiles, but yields 3
unsigned char x4 = add_sat<unsigned char>(255, 4); // ok, yields 255
unsigned char x5 = add_sat(252, x3); // error: inconsistent deductions for T
unsigned char x6 = add_sat<unsigned char>(251, x1); // ok, yields 255; might yield an int -> unsigned char conversion warning for x1
```

Design considerations

All of addition, subtraction, multiplication, and division are provided.

The operations are not defined on the integral types `bool`, `char`, `wchar_t`, `char16_t`, and `char32_t`, as these are not intended for arithmetic.

Unlike the built-in arithmetic operators on integers, this proposal expressly does not apply integral promotions to the arguments, since that would be besides the point for saturation arithmetic.

The situation for template argument deduction presented by these functions is the same as for `std::min` or `std::max`: If two arguments of different type are passed, the call fails to compile.

Instead of free functions, it is conceivable to provide an integer-like class template with the arithmetic operators suitably overloaded. This would, however, make it impossible to adopt this proposal for C, and seems slightly over-engineered for a rather simple facility.

The header `<cmath>` contains mostly (except for `abs`) floating-point functions, so integer-only arithmetic functions do not seem to fit. The header `<cstdlib>` does contain `abs` and `div` functions for integers, but its contents is mostly defined by the related C header `<stdlib.h>`, therefore I suggest to create a new header.

Regarding customization for user-defined types, these functions are considered in the same category as `std::sin` or `std::cos`. There is no intent to offer full customization point objects at this time.

Prior art

A lot of SIMD instruction sets contain CPU instructions for saturation arithmetic on SIMD vectors, including SSE2 for x86 and NEON for ARM.

A branch-free implementation for scalars is available here: https://locklessinc.com/articles/sat_arithmetic/.

Naming

Considerations:

- These are basic low-level operations. Names should be short.
- `add` / `sub` / `mul` / `div` are common abbreviation for the corresponding arithmetic operations.

Options (only showing the operation "saturated addition"):

- `addsat`, `satadd`: not easily readable
- `saturated_add`, `add_saturated`: too long
- `sat_add`: more readable due to underscore
- `add_sat`: more readable due to underscore, precedence in OpenCL

The last choice is taken.

LEWG deliberations

On 2022-08-02, LEWG discussed P0543R1. There was no consensus to introduce the saturation arithmetic functions as customization point objects. There was no consensus to change the proposed names. There was consensus to put the functions into the `<numeric>` header. There was consensus to mark the functions as freestanding. Finally, there was consensus to send the paper so modified to LWG.

Wording

In subclause 16.4.2.5 [compliance], add a row to table 27 [tab:headers.cpp.fs]:

27 [algorithms]	Algorithms library	<numeric>	
-----------------	--------------------	-----------	--

In subclause 27.9 [numeric.ops.overview], add to header `<numeric>` as indicated:

```
// 27.10.16, midpoint
template
constexpr T midpoint(T a, T b) noexcept;
template
constexpr T* midpoint(T* a, T* b);

// 27.10.17, saturation arithmetic
template<class T>
constexpr T add_sat(T x, T y) noexcept;           // freestanding
template<class T>
constexpr T sub_sat(T x, T y) noexcept;           // freestanding
template<class T>
constexpr T mul_sat(T x, T y) noexcept;           // freestanding
template<class T>
constexpr T div_sat(T x, T y) noexcept;           // freestanding
template<class T, class U>
constexpr T saturate_cast(U x) noexcept;           // freestanding
}
```

Append a new subsection to subclause 27.10 [numeric.ops] with the following content:

27.10.17 Saturation arithmetic [numeric.sat]

27.10.17.1 Arithmetic functions [numerics.sat.func]

[Note: In the following descriptions, an arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type.]

```
template<class T>
constexpr T add_sat(T x, T y) noexcept;
```

Constraints: T is a signed or unsigned integer type (6.8.2 [basic.fundamental]).

Returns: If $x + y$ is representable as a value of type T, $x + y$, otherwise either the largest or smallest representable value of type T, whichever is closer to the value of $x + y$.

```
template<class T>
constexpr T sub_sat(T x, T y) noexcept;
```

Constraints: T is a signed or unsigned integer type (6.8.2 [basic.fundamental]).

Returns: If $x - y$ is representable as a value of type T, $x - y$, otherwise either the largest or smallest representable value of type T, whichever is closer to the value of $x - y$.

```
template<class T>
constexpr T mul_sat(T x, T y) noexcept;
```

Constraints: T is a signed or unsigned integer type (6.8.2 [basic.fundamental]).

Returns: If $x * y$ is representable as a value of type T, $x * y$, otherwise either the largest or smallest representable value of type T, whichever is closer to the value of $x * y$.

```
template<class T>
constexpr T div_sat(T x, T y) noexcept;
```

Constraints: T is a signed or unsigned integer type (6.8.2 [basic.fundamental]).

Preconditions: $y \neq 0$ is true.

Let q be x / y , with any fractional part discarded.

Returns: If q is representable as a value of type T, q , otherwise either the largest or smallest representable value of type T, whichever is closer to the value of q .

Remarks: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression (7.7 [expr.const]).

27.10.17.2 Casting [numerics.sat.cast]

```
template<class T, class U>
constexpr T saturate_cast(U x) noexcept;
```

Constraints: T and U are signed or unsigned integer types (6.8.2 [basic.fundamental]).

Returns: If x is representable as a value of type T, x , otherwise either the largest or smallest representable value of type T, whichever is closer to the value of x .

Add a feature-test macro in [version.syn]:

```
#define __cpp_lib_saturation_arithmetic
```

References

- ISO/IEC JTC1 SC22 WG21 [N3864](#): "A constexpr bitwise operations library for C++" by Matthew Fioravante
- ISO/IEC JTC1 SC22 WG21 [P0214R2](#): "Data-Parallel Vector Types & Operations" by Matthias Kretz
- Wikipedia: [Saturation arithmetic](#)
- [ARM Compiler User Guide](#) [large PDF], section 13.89 "QSUB" instruction reference
- [Intel x86 Instruction Set Reference](#) [large PDF], PSUBUSB instruction